



NOTRE DAME UNIVERSITY BANGLADESH

Machine Learning Lab Report-05

Course Code: CSE4214

Course Title: Machine Learning Lab

Lab Task Topic: Naive Bayes Theorem

Submitted by:

Name: Istiak Alam

ID: 0692230005101005

Batch: CSE-20

Submission Date: April 26, 2026

Submitted to:

A. H. M. Saiful Islam

Chairman, Dept of CSE

Notre Dame University Bangladesh

Table of Contents

1	Objective	1
2	Dataset Description	1
3	Implementing Naive Bayes Theorem	1
3.1	Data Loading and Library Import for Naive Bayes	1
3.2	Encoding Categorical Variables using Label Encoding	2
3.3	Train-Test Split and Categorical Naive Bayes Model Training	3
3.4	Naive Bayes Model Prediction and Evaluation	3
3.5	Naive Bayes Prediction on New Example Data	4
4	Naive Bayes Classifier - Titanic	6
4.1	Loading Dataset using Pandas (head())	6
4.2	Handling Missing Values in Age Column	7
4.3	Dropping Irrelevant Columns from Dataset	9
4.4	Data Preparation and Encoding	9
4.5	Calculate value with cross validation	16
5	Feature-selection-stdm	16
5.1	Univariate Selecection Method @Feature Selection, Algorithm: SelectKBest Algorithm	16
6	Feature-selection-using-correlation	20
6.1	Encoding / Handling another column => location	22
7	Feature Selection using Correlation -U	23
7.1	Encoding / Handling another column => location	24
7.2	Encoding / Handling another column => area	25
7.3	Encoding more than one column at a time	25

1 Objective

The objective of this experiment is to implement the Naive Bayes classification algorithm using the `CategoricalNB` model from the Scikit-learn library. The model is used to predict whether a customer will make a purchase based on categorical features such as Day, Discount availability, and Free Delivery option. Additionally, the experiment aims to understand the process of data preprocessing, label encoding, model training, and evaluation of classification performance.

2 Dataset Description

The following datasets were used in this Naive Bayes lab experiment:

- **NaiveBayesDataset.csv**
This dataset is used for demonstrating the basic implementation of the Naive Bayes classification algorithm. It contains labeled features suitable for probability-based classification tasks.
- **titanic.csv**
This dataset contains passenger information from the Titanic disaster. It is commonly used for classification problems such as predicting survival based on features like age, gender, and passenger class.
- **cardio_train.csv**
This dataset includes medical data used to predict cardiovascular disease. It contains features such as age, blood pressure, cholesterol level, and lifestyle indicators.
- **feature_selection.csv**
This dataset is used to analyze feature importance and selection techniques. It helps in identifying the most relevant attributes for improving classification performance.

Since all features are categorical in nature, label encoding is applied to convert them into numerical form before training the Naive Bayes classifier.

3 Implementing Naive Bayes Theorem

3.1 Data Loading and Library Import for Naive Bayes

Explanation

In this step, we import essential Python libraries required for building a Naive Bayes classification model. `pandas` is used for data handling and analysis. `train_test_split` helps to divide the dataset into training and testing sets. `CategoricalNB` is the Naive Bayes classifier suitable for categorical features. `LabelEncoder` is used to convert categorical variables into numerical form. Finally, `accuracy_score` and `classification_report` are used to evaluate model performance.

After importing the libraries, the dataset `NaiveBayesDataset.csv` is loaded using `pd.read_csv()`, and displayed to verify its structure and contents.

```
[1]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import CategoricalNB
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, classification_report
```

```
[2]: # Load the dataset
df = pd.read_csv('NaiveBayesDataset.csv')
```

```
[3]: df
```

Output

The dataset is successfully loaded into a pandas DataFrame. The output displays the first view of the dataset in tabular form, showing rows and columns with categorical features and the target label. This confirms that the data is correctly imported and ready for preprocessing and model training.

```
[3]:
```

	Day	Discount	Free_Delivery	Purchase
0	Weekday	Yes	Yes	Yes
1	Weekday	Yes	Yes	Yes
2	Weekday	No	No	No
3	Holiday	Yes	Yes	Yes
4	Weekend	Yes	Yes	Yes
5	Holiday	No	No	No
6	Weekend	Yes	No	Yes
7	Weekday	Yes	Yes	Yes
8	Weekend	Yes	Yes	Yes
9	Holiday	Yes	Yes	Yes
10	Holiday	No	Yes	Yes
11	Holiday	No	No	No
12	Weekend	Yes	Yes	Yes
13	Holiday	Yes	Yes	Yes
14	Holiday	Yes	Yes	Yes
15	Weekday	Yes	Yes	Yes
16	Holiday	No	Yes	Yes
17	Weekday	Yes	No	Yes
18	Weekend	No	No	Yes
19	Weekend	No	Yes	Yes
20	Weekday	Yes	Yes	Yes
21	Weekend	Yes	Yes	No
22	Holiday	No	Yes	Yes
23	Weekday	Yes	Yes	Yes
24	Holiday	No	No	No
25	Weekday	No	Yes	No
26	Weekday	Yes	Yes	Yes
27	Weekday	Yes	Yes	Yes
28	Holiday	Yes	Yes	Yes
29	Weekend	Yes	Yes	Yes

3.2 Encoding Categorical Variables using Label Encoding

Explanation

This code converts categorical text data into numerical format using `LabelEncoder` from `sklearn`. A dictionary `label_encoders` is created to store encoders for each column. The loop iterates through selected categorical columns: `Day`, `Discount`, `Free_Delivery`, and `Purchase`. For each column, a new `LabelEncoder` object is initialized, and the categorical values are transformed into integer labels using `fit_transform()`.

```
[4]: # Encode categorical variables
label_encoders = {}
for column in ['Day', 'Discount', 'Free_Delivery', 'Purchase']:
    le = LabelEncoder()
    df[column] = le.fit_transform(df[column])
    label_encoders[column] = le
```

3.3 Train-Test Split and Categorical Naive Bayes Model Training

Explanation

In this step, the dataset is divided into feature variables and target variable. The features X include Day, Discount, and Free_Delivery, while the target variable y is Purchase, which indicates whether a customer made a purchase or not.

After defining the input and output variables, the dataset is split into training and testing sets using an 80:20 ratio. This allows the model to learn patterns from the training data and later be evaluated on unseen test data. The `random_state=42` ensures reproducibility of the split.

Next, a Categorical Naive Bayes (CategoricalNB) model is initialized. This model is specifically suitable for categorical features. The model is then trained using the training dataset (`X_train`, `y_train`) to learn the probabilistic relationship between the features and the target variable.

```
[5]: # Split data into features (X) and target (y)
X = df[['Day', 'Discount', 'Free_Delivery']]
y = df['Purchase']
```

```
[6]: # Split into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)
```

```
[7]: # Initialize and train the Naive Bayes model
nb_model = CategoricalNB()
nb_model.fit(X_train, y_train)
```

Output

After executing this code, the model does not produce a direct printed output. However, internally:

- The dataset is successfully split into training and testing sets.
- The Naive Bayes model is trained and fitted on the training data.
- The trained model is now ready to make predictions on the test set or new unseen data.

```
[7]: CategoricalNB()
```

3.4 Naive Bayes Model Prediction and Evaluation

Explanation

This code uses a trained Naive Bayes classifier (`'nb_model'`) to make predictions on the test dataset (`'X_test'`). The predicted class labels are stored in `'y_pred'`.

After prediction, the model performance is evaluated using two key metrics: - **Accuracy Score**: Measures the proportion of correctly classified samples in the test set. - **Classification Report**: Provides detailed evaluation including precision, recall, F1-score, and support for each class.

These metrics help to understand how well the model generalizes to unseen data.

```
[8]: # Make predictions on the test set
y_pred = nb_model.predict(X_test)

[9]: # Evaluate the model
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Output

The output displays: - A single accuracy value (e.g., 0.85 or 85) - A classification report table showing precision, recall, and F1-score for each class.

From this output, we can interpret whether the model performs consistently across all classes or if it is biased toward any specific class.

Accuracy: 1.0

	precision	recall	f1-score	support
1	1.00	1.00	1.00	6
accuracy			1.00	6
macro avg	1.00	1.00	1.00	6
weighted avg	1.00	1.00	1.00	6

3.5 Naive Bayes Prediction on New Example Data

Explanation

This code demonstrates how a trained Naive Bayes model is used to make a prediction on a new unseen sample. The input example consists of categorical features such as Day, Discount, and Free_Delivery. Since machine learning models cannot directly process categorical text values, each feature is first converted into numerical form using previously fitted LabelEncoder objects.

After encoding, the processed data is passed into the trained Naive Bayes model (nb_model) to predict whether a customer will make a purchase or not. The predicted numerical output is then converted back into its original categorical label (e.g., "Yes" or "No") using inverse_transform() for better interpretability.

Finally, the code also prints the encoded classes for the Day feature to verify what categories the encoder has learned during training.

Code Logic Summary:

- Create a new input sample as a DataFrame
- Encode categorical variables using label encoders
- Predict output using trained Naive Bayes model
- Convert prediction back to original label

- Display encoded classes of a feature

Output

The output will display:

- The predicted purchase decision (e.g., "Purchase Prediction: Yes" or "No")
- The list of encoded classes for the *Day* feature (e.g., Holiday, Weekend, Weekday)

This helps verify both the model's prediction and the encoding structure used during preprocessing.

```
[10]: # Example prediction
example = pd.DataFrame({'Day': ['Holiday'], 'Discount': ['Yes'],
    ↪ 'Free_Delivery': ['No']})
for column in example.columns:
    example[column] = label_encoders[column].transform(example[column])
prediction = nb_model.predict(example)
print("Purchase Prediction:", label_encoders['Purchase'].
    ↪ inverse_transform(prediction))
```

Purchase Prediction: ['Yes']

```
[11]: # Check if "Holiday" exists in the Day encoder classes
print("Classes for Day:", label_encoders['Day'].classes_)
```

Classes for Day: ['Holiday' 'Weekday' 'Weekend']

```
[12]: # Example prediction
example = pd.DataFrame({'Day': ['Holiday'], 'Discount': ['Yes'],
    ↪ 'Free_Delivery': ['No']})
# This line creates a DataFrame named example with a single row representing
    ↪ an example input.
```

```
[13]: # Encode the example input using updated encoders
for column in example.columns:
    example[column] = label_encoders[column].transform(example[column])
#Purpose: This loop iterates over each column in the example DataFrame and
    ↪ transforms the categorical
#text values into numeric codes, which the model requires as input.
```

```
[14]: # Make prediction
prediction = nb_model.predict(example)
#Purpose: This line uses the Naive Bayes model (nb_model) to make a
    ↪ prediction
# on the transformed example input.
```

```
[15]: # Decode the prediction back to original label
print("Purchase Prediction:", label_encoders['Purchase'].
    ↪ inverse_transform(prediction))
```

Purchase Prediction: ['Yes']

```
[16]: # Example prediction
example = pd.DataFrame({'Day': ['Weekend'], 'Discount': ['Yes'],
    ↪ 'Free_Delivery': ['Yes']})
# This line creates a DataFrame named example with a single row representing
    ↪ an example input

[17]: # Encode the example input using updated encoders
for column in example.columns:
    example[column] = label_encoders[column].transform(example[column])
#Purpose: This loop iterates over each column in the example DataFrame and
    ↪ transforms the categorical
#text values into numeric codes, which the model requires as input.

[18]: # Make prediction
prediction = nb_model.predict(example)
#Purpose: This line uses the Naive Bayes model (nb_model) to make a
    ↪ prediction
# on the transformed example input.

[19]: # Decode the prediction back to original label
print("Purchase Prediction:", label_encoders['Purchase'].
    ↪ inverse_transform(prediction))
```

Purchase Prediction: ['Yes']

4 Naive Bayes Classifier - Titanic

4.1 Loading Dataset using Pandas (head())

Explanation

In this code, we import the Pandas library and load a CSV file named `titanic.csv` into a `DataFrame` using `pd.read_csv()`. After loading the dataset, we use the `head()` function to display the first five rows of the dataset. This is commonly used to quickly inspect the structure, column names, and sample data of a dataset before performing further analysis or preprocessing.

```
[20]: import pandas as pd
```

```
[21]: df= pd.read_csv('titanic.csv')
df.head()
```

```
[21]: PassengerId  Survived  Pclass  \
0            892         0       3
1            893         1       3
2            894         0       2
3            895         0       3
4            896         1       3

                                     Name      Sex  Age  SibSp  Parch  \
    ↪\
0                                Kelly, Mr. James   male  34.5     0     0
1                Wilkes, Mrs. James (Ellen Needs) female  47.0     1     0
```

2	Myles, Mr. Thomas Francis	male	62.0	0	0
3	Wirz, Mr. Albert	male	27.0	0	0
4	Hirvonen, Mrs. Alexander (Helga E Lindqvist)	female	22.0	1	1

	Ticket	Fare	Cabin	Embarked
0	330911	7.8292	NaN	Q
1	363272	7.0000	NaN	S
2	240276	9.6875	NaN	Q
3	315154	8.6625	NaN	S
4	3101298	12.2875	NaN	S

```
[22]: df.isnull().sum()
```

Output

The output displays the first 5 rows of the Titanic dataset. It typically includes columns such as PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, and Embarked. Each row represents a passenger record, allowing a quick preview of the dataset's structure and values.

```
[22]: PassengerId    0
      Survived      0
      Pclass       0
      Name         0
      Sex          0
      Age         86
      SibSp        0
      Parch        0
      Ticket       0
      Fare         1
      Cabin       327
      Embarked     0
      dtype: int64
```

4.2 Handling Missing Values in Age Column

Explanation

This code handles missing (null) values in the Age column of the dataset using mean imputation. First, the mean value of the Age column is calculated and stored in the variable `handle`. Then, all null values in Age are replaced with this mean value using the `fillna()` function. This is a common preprocessing technique to preserve dataset size while maintaining statistical consistency.

```
[23]: # removing null values of Age// better if we remove later on the updated
      ↪ dataset
      handle = df['Age'].mean()
      handle
```

Output

The output of this operation is the computed mean of the Age column (stored in `handle`). After execution, the Age column in the dataframe will no longer contain any missing values, as all NaNs

are replaced with the calculated mean value.

```
[23]: np.float64(30.272590361445783)
```

```
[24]: df.Age= df.Age.fillna(handle)
```

```
[25]: df
```

```
[25]:
```

	PassengerId	Survived	Pclass	\
0	892	0	3	
1	893	1	3	
2	894	0	2	
3	895	0	3	
4	896	1	3	
..	...	...	...	
413	1305	0	3	
414	1306	1	1	
415	1307	0	3	
416	1308	0	3	
417	1309	0	3	

	Name	Sex	Age	SibSp	\
0	Kelly, Mr. James	male	34.50000	0	
1	Wilkes, Mrs. James (Ellen Needs)	female	47.00000	1	
2	Myles, Mr. Thomas Francis	male	62.00000	0	
3	Wirz, Mr. Albert	male	27.00000	0	
4	Hirvonen, Mrs. Alexander (Helga E Lindqvist)	female	22.00000	1	
..	...	...	...	...	
413	Spector, Mr. Woolf	male	30.27259	0	
414	Oliva y Ocana, Dona. Fermina	female	39.00000	0	
415	Saether, Mr. Simon Sivertsen	male	38.50000	0	
416	Ware, Mr. Frederick	male	30.27259	0	
417	Peter, Master. Michael J	male	30.27259	1	

	Parch	Ticket	Fare	Cabin	Embarked
0	0	330911	7.8292	NaN	Q
1	0	363272	7.0000	NaN	S
2	0	240276	9.6875	NaN	Q
3	0	315154	8.6625	NaN	S
4	1	3101298	12.2875	NaN	S
..	...	...	...	...	...
413	0	A.5. 3236	8.0500	NaN	S
414	0	PC 17758	108.9000	C105	C
415	0	SOTON/O.Q. 3101262	7.2500	NaN	S
416	0	359309	8.0500	NaN	S
417	1	2668	22.3583	NaN	C

```
[418 rows x 12 columns]
```

4.3 Dropping Irrelevant Columns from Dataset

Explanation

In this step, unnecessary features are removed from the dataset to simplify the model and reduce noise. The selected columns such as PassengerId, Name, SibSp, Parch, Ticket, Cabin, and Embarked are dropped because they are assumed to have weak or no direct influence on the target variable under the naive independence assumption. This helps improve model efficiency and focuses learning on more relevant features.

```
[26]: # Assumption: Make a naive assumption that features such as
      ↪ male, class, age, cabin, fare, etc. are independant of each other

[27]: df.
      ↪ drop(['PassengerId', 'Name', 'SibSp', 'Parch', 'Ticket', 'Cabin', 'Embarked'], axis=
      ↪ 'columns', inplace= True)

[28]: df.head()
```

Output

After executing the code, the specified columns are permanently removed from the dataframe using `inplace=True`. The command `df.head()` then displays the first five rows of the updated dataset, showing only the remaining relevant features such as survival-related attributes (e.g., class, sex, age, fare). The output confirms that the dataframe structure has been successfully reduced and cleaned for further analysis or model training.

```
[28]:   Survived  Pclass    Sex   Age   Fare
0         0       3   male  34.5   7.8292
1         1       3 female  47.0   7.0000
2         0       2   male  62.0   9.6875
3         0       3   male  27.0   8.6625
4         1       3 female  22.0  12.2875
```

4.4 Data Preparation and Encoding

Explanation

In this step, the dataset is prepared for model training. The Survived column is separated as the target variable, where 1 indicates survived and 0 indicates not survived. The remaining columns are treated as input features by dropping the Survived column.

Next, the categorical column Sex is converted into numerical form using one-hot encoding with `pd.get_dummies()`. This creates separate columns (e.g., male and female) with binary values. The values are then converted from boolean (True/False) to integer (1/0) for better compatibility with machine learning models.

```
[29]: # 1 - survived 0 - not survived , At the end we will work on final
      # obtained from inputs and targer for train_test_split()

[30]: # 1 - survived 0 - not survived
      inputs= df.drop('Survived', axis = 'columns')
      target = df.Survived
```

```
[31]: # Encoding Sex
dummies = pd.get_dummies(df['Sex'])
```

```
[32]: dummies = dummies.astype(int)
# will display 0 and 1 instead of True and False
```

```
[33]: dummies
```

Output

The output consists of a new dataframe containing encoded columns for the Sex feature. Each row will have values 0 or 1 indicating the presence of a category (male or female), instead of True/False.

```
[33]:
```

	female	male
0	0	1
1	1	0
2	0	1
3	0	1
4	1	0
..	...	...
413	0	1
414	1	0
415	0	1
416	0	1
417	0	1

[418 rows x 2 columns]

```
[34]: #inputs = inputs.drop('Sex', 'male' axis='columns')
```

```
[35]: df
```

```
[35]:
```

	Survived	Pclass	Sex	Age	Fare
0	0	3	male	34.50000	7.8292
1	1	3	female	47.00000	7.0000
2	0	2	male	62.00000	9.6875
3	0	3	male	27.00000	8.6625
4	1	3	female	22.00000	12.2875
..	...	...	...	...	...
413	0	3	male	30.27259	8.0500
414	1	1	female	39.00000	108.9000
415	0	3	male	38.50000	7.2500
416	0	3	male	30.27259	8.0500
417	0	3	male	30.27259	22.3583

[418 rows x 5 columns]

```
[36]: inputs
```

```
[36]:
```

	Pclass	Sex	Age	Fare
0	3	male	34.50000	7.8292
1	3	female	47.00000	7.0000

```

2          2    male  62.00000    9.6875
3          3    male  27.00000    8.6625
4          3  female  22.00000   12.2875
..      ...      ...      ...      ...
413        3    male  30.27259    8.0500
414        1  female  39.00000  108.9000
415        3    male  38.50000    7.2500
416        3    male  30.27259    8.0500
417        3    male  30.27259   22.3583

```

[418 rows x 4 columns]

```
[37]: inputs = inputs.drop(['Sex'],axis='columns')
```

```
[38]: inputs
```

```

[38]:      Pclass      Age      Fare
0         3  34.50000    7.8292
1         3  47.00000    7.0000
2         2  62.00000    9.6875
3         3  27.00000    8.6625
4         3  22.00000   12.2875
..      ...      ...      ...
413        3  30.27259    8.0500
414        1  39.00000  108.9000
415        3  38.50000    7.2500
416        3  30.27259    8.0500
417        3  30.27259   22.3583

```

[418 rows x 3 columns]

```
[39]: #concatening columns with sex column
df
```

```

[39]:      Survived  Pclass      Sex      Age      Fare
0           0         3    male  34.50000    7.8292
1           1         3  female  47.00000    7.0000
2           0         2    male  62.00000    9.6875
3           0         3    male  27.00000    8.6625
4           1         3  female  22.00000   12.2875
..      ...      ...      ...      ...      ...
413          0         3    male  30.27259    8.0500
414          1         1  female  39.00000  108.9000
415          0         3    male  38.50000    7.2500
416          0         3    male  30.27259    8.0500
417          0         3    male  30.27259   22.3583

```

[418 rows x 5 columns]

```
[40]: inputs
```

```
[40]:
```

	Pclass	Age	Fare
0	3	34.50000	7.8292
1	3	47.00000	7.0000
2	2	62.00000	9.6875
3	3	27.00000	8.6625
4	3	22.00000	12.2875
..	...	...	...
413	3	30.27259	8.0500
414	1	39.00000	108.9000
415	3	38.50000	7.2500
416	3	30.27259	8.0500
417	3	30.27259	22.3583

[418 rows x 3 columns]

```
[41]: merged = pd.concat([inputs,dummies],axis = 'columns')
```

```
[42]: merged
```

```
[42]:
```

	Pclass	Age	Fare	female	male
0	3	34.50000	7.8292	0	1
1	3	47.00000	7.0000	1	0
2	2	62.00000	9.6875	0	1
3	3	27.00000	8.6625	0	1
4	3	22.00000	12.2875	1	0
..	...	...	...	...	...
413	3	30.27259	8.0500	0	1
414	1	39.00000	108.9000	1	0
415	3	38.50000	7.2500	0	1
416	3	30.27259	8.0500	0	1
417	3	30.27259	22.3583	0	1

[418 rows x 5 columns]

```
[43]: final = merged.drop(['male'],axis = 'columns')
```

```
[44]: final
```

```
[44]:
```

	Pclass	Age	Fare	female
0	3	34.50000	7.8292	0
1	3	47.00000	7.0000	1
2	2	62.00000	9.6875	0
3	3	27.00000	8.6625	0
4	3	22.00000	12.2875	1
..	...	...	...	...
413	3	30.27259	8.0500	0
414	1	39.00000	108.9000	1
415	3	38.50000	7.2500	0
416	3	30.27259	8.0500	0
417	3	30.27259	22.3583	0

```
[418 rows x 4 columns]
```

```
[45]: final.columns[final.isna().any()]
```

```
[45]: Index(['Fare'], dtype='str')
```

```
[46]: null_count_city = df['Fare'].isnull().sum()  
print(f"Number of null values in 'Fare' column: {null_count_city}")
```

```
Number of null values in 'Fare' column: 1
```

```
[47]: final.Fare[:50]
```

```
[47]: 0      7.8292  
1      7.0000  
2      9.6875  
3      8.6625  
4     12.2875  
5      9.2250  
6      7.6292  
7     29.0000  
8      7.2292  
9     24.1500  
10     7.8958  
11     26.0000  
12     82.2667  
13     26.0000  
14     61.1750  
15     27.7208  
16     12.3500  
17      7.2250  
18      7.9250  
19      7.2250  
20     59.4000  
21      3.1708  
22     31.6833  
23     61.3792  
24    262.3750  
25     14.5000  
26     61.9792  
27      7.2250  
28     30.5000  
29     21.6792  
30     26.0000  
31     31.5000  
32     20.5750  
33     23.4500  
34     57.7500  
35      7.2292  
36      8.0500  
37      8.6625  
38      9.5000
```

```

39      56.4958
40      13.4167
41      26.5500
42       7.8500
43      13.0000
44      52.5542
45       7.9250
46      29.7000
47       7.7500
48      76.2917
49      15.9000
Name: Fare, dtype: float64

```

```
[48]: final.Fare = final.Fare.fillna(final.Fare.mean())
final.head()
```

```
[48]:
```

	Pclass	Age	Fare	female
0	3	34.5	7.8292	0
1	3	47.0	7.0000	1
2	2	62.0	9.6875	0
3	3	27.0	8.6625	0
4	3	22.0	12.2875	1

```
[49]: from sklearn.model_selection import train_test_split
```

```
[50]: xtrain, xtest, ytrain, ytest = train_test_split(final, target, test_size = 0.
↳30, random_state =1)
```

```
[51]: from sklearn.naive_bayes import GaussianNB
model = GaussianNB ()
```

```
[52]: model.fit(xtrain, ytrain)
```

```
[52]: GaussianNB()
```

```
[53]: model.score(xtest,ytest)
```

```
[53]: 1.0
```

```
[54]: xtest[:10]
```

```
[54]:
```

	Pclass	Age	Fare	female
358	3	30.27259	7.7500	0
164	2	41.00000	13.0000	0
17	3	21.00000	7.2250	0
67	1	47.00000	42.4000	0
4	3	22.00000	12.2875	1
377	2	21.00000	11.5000	0
214	3	38.00000	7.7750	1
290	1	30.27259	39.6000	0
381	3	26.00000	7.8792	0

```
5          3  14.00000  9.2250      0
```

```
[55]: ytest[0:10]
```

```
[55]: 358    0
      164    0
      17    0
      67    0
       4    1
      377    0
      214    1
      290    0
      381    0
       5    0
      Name: Survived, dtype: int64
```

```
[56]: model.predict(xtest[0:10])
```

```
[56]: array([0, 0, 0, 0, 1, 0, 1, 0, 0, 0])
```

```
[57]: model.predict_proba(xtest[:10])
```

```
[57]: array([[1., 0.],
             [1., 0.],
             [1., 0.],
             [1., 0.],
             [0., 1.],
             [1., 0.],
             [0., 1.],
             [1., 0.],
             [1., 0.],
             [1., 0.]])
```

```
[58]: #pclass, Age, Fare, Gender
```

```
test = [[1,23.000000,113.2750,0]]
#test = xtest
a= model.predict(test)
if a[0] == 0:
    print("Not Survived")
else:
    print(" Survived")
```

Not Survived

```
/media/spyder/Secondary-Disk/Github/Student-Performance-Prediction-with-
Explainable-AI/CodeBase/Trial/.spenv/lib/python3.11/site-
packages/sklearn/base.py:493: UserWarning: X does not have valid feature
  names,
but GaussianNB was fitted with feature names
  warnings.warn(
```

4.5 Calculate value with cross validation

```
[59]: from sklearn.model_selection import cross_val_score
```

```
[60]: cross_val_score (GaussianNB(),xtrain,ytrain,cv = 5)
```

```
[60]: array([1., 1., 1., 1., 1.])
```

5 Feature-selection-stdm

5.1 Univariate Selecection Method @Feature Selection, Algorithm: SelectKBest Algorithm

```
[61]: import pandas as pd
import numpy as np
import seaborn as sns
```

```
[62]: #df = pd.read_csv("bank.csv",sep = ';')
df = pd.read_csv("cardio_train.csv", sep = ';')
```

```
[63]: df
```

```
[63]:
```

	id	age	gender	height	weight	ap_hi	ap_lo	cholesterol	gluc
0	0	18393	2	168	62.0	110	80	1	1
1	1	20228	1	156	85.0	140	90	3	1
2	2	18857	1	165	64.0	130	70	3	1
3	3	17623	2	169	82.0	150	100	1	1
4	4	17474	1	156	56.0	100	60	1	1
...	...	...	...	...	...	...	...	...	...
69995	99993	19240	2	168	76.0	120	80	1	1
69996	99995	22601	1	158	126.0	140	90	2	2
69997	99996	19066	2	183	105.0	180	90	3	1
69998	99998	22431	1	163	72.0	135	80	1	2
69999	99999	20540	1	170	72.0	120	80	2	1

	smoke	alco	active	cardio
0	0	0	1	0
1	0	0	1	1
2	0	0	0	1
3	0	0	1	1
4	0	0	0	0
...	...	...	...	...
69995	1	0	1	0
69996	0	0	1	1
69997	0	1	0	1
69998	0	0	0	1
69999	0	0	1	0

```
[70000 rows x 13 columns]
```

```
[64]: df.shape
```

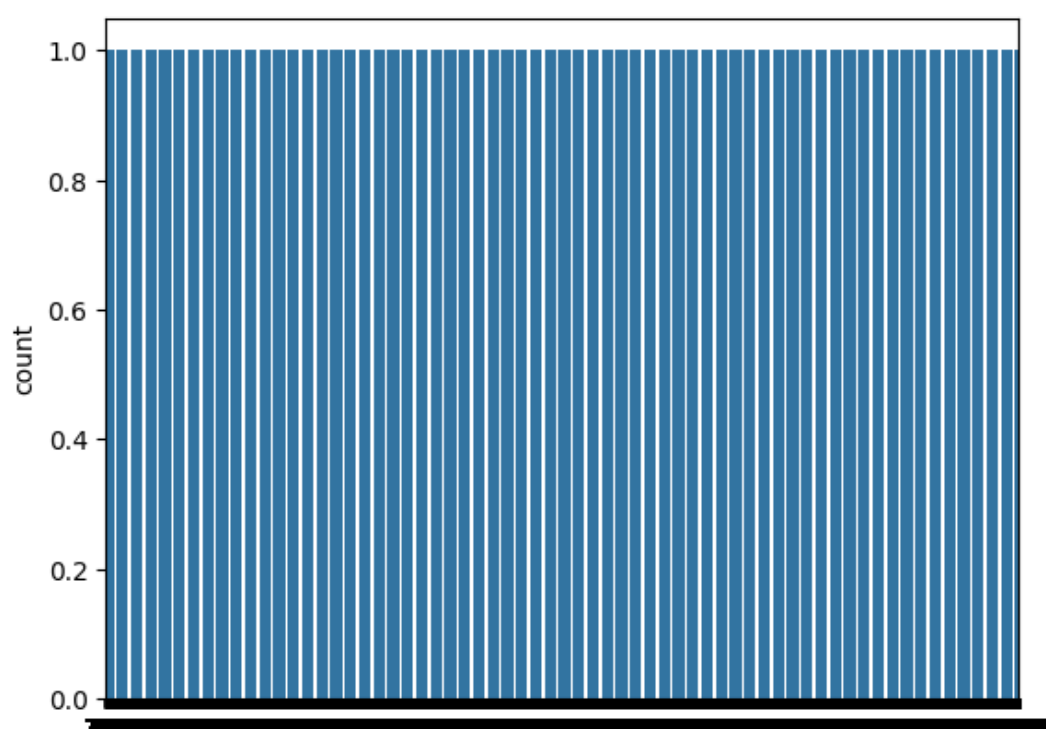
```
[64]: (70000, 13)
```

```
[65]: df["cardio"].value_counts()
```

```
[65]: cardio  
0    35021  
1    34979  
Name: count, dtype: int64
```

```
[66]: sns.countplot(df["cardio"])
```

```
[66]: <Axes: ylabel='count'>
```



- `x = df.iloc[:, :-1]:`
- This line is selecting all rows (`:`) and all columns except the last one (`:-1`).
- `:` means “select all rows.”
- `:-1` means “select all columns except the last one.” In Python slicing, `:-1` excludes the last item, so this will include all columns up to, but not including, the last one.
- Result: `x` will be a DataFrame containing all the columns of `df` except the last column

```
[67]: x = df.iloc[:, :-1]
```

- `y = df.iloc[:, 12]`
- This line is selecting all rows (`:`) and the 13th column (remember, indexing in Python starts at 0, so index 12 corresponds to the 13th column).
- `:` means “select all rows.”

- 12 means “select the 13th column” (as Python uses 0-based indexing).
- Result: y will be a Series containing the values of the 13th column of df.

```
[68]: # y = df.iloc[:, 12]
y = df.iloc[:, 12]
```

```
[69]: from sklearn.feature_selection import SelectKBest
```

```
[70]: from sklearn.feature_selection import f_classif
```

```
[71]: FIT_FEATURES = SelectKBest(score_func = f_classif)
```

```
[72]: FIT_FEATURES.fit(x, y)
```

```
[72]: SelectKBest()
```

```
[73]: pd.DataFrame(FIT_FEATURES.scores_)
```

```
[73]:
0      0
0      1.010461
1      4209.007957
2      4.603641
3      8.197397
4      2388.777887
5      208.339524
6      303.629011
7      3599.361137
8      562.772977
9      16.790541
10     3.761355
11     89.091494
```

```
[74]: score_COL = pd.DataFrame(FIT_FEATURES.scores_)
```

```
[75]: score_COL
```

```
[75]:
0      0
0      1.010461
1      4209.007957
2      4.603641
3      8.197397
4      2388.777887
5      208.339524
6      303.629011
7      3599.361137
8      562.772977
9      16.790541
10     3.761355
11     89.091494
```

```
[76]: # Assuming FIT_FEATURES_scores is an array or a list of scores
score_COL = pd.DataFrame(FIT_FEATURES.scores_, columns=["scorevalue"])
```

```
# Now score_COL is a DataFrame with a single column named "scorevalue"
```

```
[77]: score_COL
```

```
[77]:      scorevalue
0         1.010461
1    4209.007957
2         4.603641
3         8.197397
4    2388.777887
5        208.339524
6        303.629011
7    3599.361137
8        562.772977
9        16.790541
10        3.761355
11       89.091494
```

```
[78]: NEW_COL = pd.DataFrame(x.columns)
```

```
[79]: NEW_COL
```

```
[79]:      0
0      id
1      age
2     gender
3     height
4     weight
5     ap_hi
6     ap_lo
7  cholesterol
8         gluc
9        smoke
10        alco
11       active
```

```
[80]: top_features = pd.concat([NEW_COL, score_COL], axis=1)
```

```
[81]: top_features
```

```
[81]:      0      scorevalue
0      id         1.010461
1      age    4209.007957
2     gender         4.603641
3     height         8.197397
4     weight    2388.777887
5     ap_hi        208.339524
6     ap_lo        303.629011
7  cholesterol    3599.361137
8         gluc        562.772977
9        smoke        16.790541
```

```
10      alco      3.761355
11      active    89.091494
```

```
[82]: top_features.nlargest (8,"scorevalue")
```

```
[82]:          0  scorevalue
1      age  4209.007957
7  cholesterol  3599.361137
4      weight  2388.777887
8      gluc   562.772977
6      ap_lo   303.629011
5      ap_hi   208.339524
11     active   89.091494
9      smoke   16.790541
```

6 Feature-selection-using-correlation

```
[83]: import pandas as pd
      # Load the dataset
      df = pd.read_csv('feature_selection.csv')
```

```
[84]: df
```

```
[84]:  location      area  bedroom  bathroom  owner  blodd group  price
0      a      dhaka         3          2      x      o+  10000
1      b      dhaka         4          2      y      ab+  12000
2      c    khulna         2          2      p      ab-  11000
3      d  rajshahi         3          2      q      o+  15000
4      e    cumilla         3          1      w      ab+  12000
5      f      dhaka         2          1      l      ab-  11000
6      g  barishal         2          1      r      o+  10000
7      h      natore         2          3      s      o+  15000
8      i    khulna         3          3      k      ab+  12000
9      j      dhaka         4          2      v      ab-  11000
```

```
[85]: # Check for non-numeric data in the columns
      print(df.dtypes)
```

```
location      str
area          str
bedroom       int64
bathroom      int64
owner         str
blodd group   str
price         int64
dtype: object
```

```
[86]: # Identify non-numeric values in specific columns
      for column in df.columns:
          if df[column].dtype == 'object':
              print(f"Non-numeric values in '{column}':\n", df[column].unique())
```

```
[87]: # Option 1: Convert columns to numeric, forcing errors to NaN
df = df.apply(pd.to_numeric, errors='coerce')
```

```
[88]: df
```

```
[88]:
```

	location	area	bedroom	bathroom	owner	blodd group	price
0	NaN	NaN	3	2	NaN	NaN	10000
1	NaN	NaN	4	2	NaN	NaN	12000
2	NaN	NaN	2	2	NaN	NaN	11000
3	NaN	NaN	3	2	NaN	NaN	15000
4	NaN	NaN	3	1	NaN	NaN	12000
5	NaN	NaN	2	1	NaN	NaN	11000
6	NaN	NaN	2	1	NaN	NaN	10000
7	NaN	NaN	2	3	NaN	NaN	15000
8	NaN	NaN	3	3	NaN	NaN	12000
9	NaN	NaN	4	2	NaN	NaN	11000

```
[89]: # Calculate correlation with the target variable 'price'
correlation = df.corr()
```

```
[90]: print(df.dtypes)
```

```
location      float64
area          float64
bedroom       int64
bathroom      int64
owner         float64
blodd group   float64
price         int64
dtype: object
```

```
[91]: # Option 1: Convert columns to numeric, forcing errors to NaN
df = df.apply(pd.to_numeric, errors='coerce')
```

```
[92]: # Calculate correlation with the target variable 'price'
correlation = df.corr()
```

```
[93]: # Display the correlation of each feature with the target variable 'price'
correlated_features = correlation['price'].sort_values(ascending=False)
print("Correlation with Price:\n", correlated_features)
```

```
Correlation with Price:
price      1.000000
bathroom   0.495798
bedroom    -0.015721
location   NaN
area       NaN
owner      NaN
blodd group NaN
Name: price, dtype: float64
```

6.1 Encoding / Handling another column => location

```
[94]: from sklearn.preprocessing import OrdinalEncoder
      # Load the dataset
      df = pd.read_csv('feature_selection.csv')
```

```
[95]: categorical_column = 'location'
```

```
[96]: df
```

```
[96]:
```

	location	area	bedroom	bathroom	owner	blodd group	price
0	a	dhaka	3	2	x	o+	10000
1	b	dhaka	4	2	y	ab+	12000
2	c	khulna	2	2	p	ab-	11000
3	d	rajshahi	3	2	q	o+	15000
4	e	cumilla	3	1	w	ab+	12000
5	f	dhaka	2	1	l	ab-	11000
6	g	barishal	2	1	r	o+	10000
7	h	natore	2	3	s	o+	15000
8	i	khulna	3	3	k	ab+	12000
9	j	dhaka	4	2	v	ab-	11000

```
[97]: print(df.columns)
```

```
Index(['location ', 'area ', 'bedroom ', 'bathroom', 'owner ', 'blodd group',
      'price'],
      dtype='str')
```

```
[98]: df.columns = df.columns.str.strip() # Remove any leading/trailing spaces
      df.columns = df.columns.str.lower() # Convert all column names to lowercase
      ↪ (optional)
      print(df.columns)
```

```
Index(['location', 'area', 'bedroom', 'bathroom', 'owner', 'blodd group',
      'price'],
      dtype='str')
```

```
[99]: # Optional: Use a conditional check
      if 'location' in df.columns:
          print(df['location'].head())
      else:
          print("The 'location' column is not in the DataFrame.")
```

```
0    a
1    b
2    c
3    d
4    e
Name: location, dtype: str
```

```
[100]: encoder = OrdinalEncoder()
        df[categorical_column] = encoder.fit_transform(df[[categorical_column]])
```

```
[101]: df
```

```
[101]:
```

	location	area	bedroom	bathroom	owner	blodd	group	price
0	0.0	dhaka	3	2	x		o+	10000
1	1.0	dhaka	4	2	y		ab+	12000
2	2.0	khulna	2	2	p		ab-	11000
3	3.0	rajshahi	3	2	q		o+	15000
4	4.0	cumilla	3	1	w		ab+	12000
5	5.0	dhaka	2	1	l		ab-	11000
6	6.0	barishal	2	1	r		o+	10000
7	7.0	natore	2	3	s		o+	15000
8	8.0	khulna	3	3	k		ab+	12000
9	9.0	dhaka	4	2	v		ab-	11000

7 Feature Selection using Correlation -U

```
[102]: import pandas as pd
# Load the dataset
df = pd.read_csv('feature_selection.csv')
```

```
[103]: df
```

```
[103]:
```

	location	area	bedroom	bathroom	owner	blodd	group	price
0	a	dhaka	3	2	x		o+	10000
1	b	dhaka	4	2	y		ab+	12000
2	c	khulna	2	2	p		ab-	11000
3	d	rajshahi	3	2	q		o+	15000
4	e	cumilla	3	1	w		ab+	12000
5	f	dhaka	2	1	l		ab-	11000
6	g	barishal	2	1	r		o+	10000
7	h	natore	2	3	s		o+	15000
8	i	khulna	3	3	k		ab+	12000
9	j	dhaka	4	2	v		ab-	11000

```
[104]: # Check for non-numeric data in the columns
print(df.dtypes)
```

```
location      str
area          str
bedroom       int64
bathroom      int64
owner         str
blodd group   str
price         int64
dtype: object
```

```
[105]: # Identify non-numeric values in specific columns
for column in df.columns:
    if df[column].dtype == 'object':
        print(f"Non-numeric values in '{column}':\n", df[column].unique())
```

7.1 Encoding / Handling another column => location

```
[106]: from sklearn.preprocessing import OrdinalEncoder
categorical_column = 'location'
df
```

```
[106]:
```

	location	area	bedroom	bathroom	owner	blodd group	price
0	a	dhaka	3	2	x	o+	10000
1	b	dhaka	4	2	y	ab+	12000
2	c	khulna	2	2	p	ab-	11000
3	d	rajshahi	3	2	q	o+	15000
4	e	cumilla	3	1	w	ab+	12000
5	f	dhaka	2	1	l	ab-	11000
6	g	barishal	2	1	r	o+	10000
7	h	natore	2	3	s	o+	15000
8	i	khulna	3	3	k	ab+	12000
9	j	dhaka	4	2	v	ab-	11000

```
[107]: print(df.columns)

Index(['location ', 'area ', 'bedroom ', 'bathroom', 'owner ', 'blodd group',
      'price'],
      dtype='str')
```

```
[108]: df.columns = df.columns.str.strip() # Remove any leading/trailing spaces
df.columns = df.columns.str.lower() # Convert all column names to lowercase
↪ (o
print(df.columns)

Index(['location', 'area', 'bedroom', 'bathroom', 'owner', 'blodd group',
      'price'],
      dtype='str')
```

```
[109]: # Optional: Use a conditional check
if 'location' in df.columns:
    print(df['location'].head())
else:
    print("The 'location' column is not in the DataFrame.")
```

```
0    a
1    b
2    c
3    d
4    e
Name: location, dtype: str
```

```
[110]: encoder = OrdinalEncoder()
df[categorical_column] = encoder.fit_transform(df[[categorical_column]])
```

```
[111]: df
```

```
[111]:
```

	location	area	bedroom	bathroom	owner	blodd group	price
0	0.0	dhaka	3	2	x	o+	10000

1	1.0	dhaka	4	2	y	ab+	12000
2	2.0	khulna	2	2	p	ab-	11000
3	3.0	rajshahi	3	2	q	o+	15000
4	4.0	cumilla	3	1	w	ab+	12000
5	5.0	dhaka	2	1	l	ab-	11000
6	6.0	barishal	2	1	r	o+	10000
7	7.0	natore	2	3	s	o+	15000
8	8.0	khulna	3	3	k	ab+	12000
9	9.0	dhaka	4	2	v	ab-	11000

7.2 Encoding / Handling another column => area

```
[112]: categorical_column = 'area'
```

```
[113]: encoder = OrdinalEncoder()
df[categorical_column] = encoder.fit_transform(df[[categorical_column]])
```

```
[114]: df
```

```
[114]:
```

	location	area	bedroom	bathroom	owner	blodd	group	price
0	0.0	2.0	3	2	x		o+	10000
1	1.0	2.0	4	2	y		ab+	12000
2	2.0	3.0	2	2	p		ab-	11000
3	3.0	5.0	3	2	q		o+	15000
4	4.0	1.0	3	1	w		ab+	12000
5	5.0	2.0	2	1	l		ab-	11000
6	6.0	0.0	2	1	r		o+	10000
7	7.0	4.0	2	3	s		o+	15000
8	8.0	3.0	3	3	k		ab+	12000
9	9.0	2.0	4	2	v		ab-	11000

7.3 Encoding more than one column at a time

```
[115]: categorical_columns = ['owner', 'blodd group']
```

```
[116]: for column in categorical_columns:
        if column not in df.columns:
            print(f"Column '{column}' does not exist in the DataFrame")
        else:
            print(f"Encoding column: {column}")
```

Encoding column: owner

Encoding column: blodd group

```
[117]: encoder = OrdinalEncoder()
existing_categorical_columns = [col for col in categorical_columns if col in
    ↪df]
df[existing_categorical_columns] = encoder.
    ↪fit_transform(df[existing_categorical_columns])
```

```
[118]: df
```

```
[118]:
```

	location	area	bedroom	bathroom	owner	blodd	group	price
0	0.0	2.0	3	2	8.0		2.0	10000
1	1.0	2.0	4	2	9.0		0.0	12000
2	2.0	3.0	2	2	2.0		1.0	11000
3	3.0	5.0	3	2	3.0		2.0	15000
4	4.0	1.0	3	1	7.0		0.0	12000
5	5.0	2.0	2	1	1.0		1.0	11000
6	6.0	0.0	2	1	4.0		2.0	10000
7	7.0	4.0	2	3	5.0		2.0	15000
8	8.0	3.0	3	3	0.0		0.0	12000
9	9.0	2.0	4	2	6.0		1.0	11000

```
[119]: # Calculate correlation with the target variable 'price'
correlation = df.corr()
```

```
[120]: # Display the correlation of each feature with the target variable 'price'
correlated_features = correlation['price'].sort_values(ascending=False)
print("Correlation with Price:\n", correlated_features)
```

```
Correlation with Price:
price      1.000000
area       0.797921
bathroom   0.495798
blodd group 0.148712
location   0.133118
bedroom    -0.015721
owner      -0.092159
Name: price, dtype: float64
```